VITyarthi | Open Source Software

THE OPEN SOURCE AUDIT

A Capstone Project for the OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5 | Max Marks: 100

Student Name	Registration Number
Amogh Patel	**24BAI10148**
Slot	Date of Submission
A13	**31 / 03 / 2026**

1. Introduction

Most of the software that powers the modern world was not built by a single company and sold for profit — it was built openly, shared freely, and improved collectively. This project is an audit of one such foundational tool: Git.

This report explores Git's origin, the philosophy of its creator, the specific freedoms guaranteed by its license, and its broader impact on the global software ecosystem. Alongside theoretical analysis, practical Linux footprint mapping and shell scripting tasks demonstrate how automation and command-line mastery connect directly to the open-source ethos of transparency.

Understanding these foundations is not an optional academic exercise — it is a prerequisite for modern professional software development. Every line of code written in a career will sit on top of open-source foundations.

2. Software Selection

The following approved software options were available for this audit:

Software	Category	License	Why It's Interesting
Linux Kernel	Operating System	GPL v2	The foundation everything else runs on
Apache HTTP Server	Web Server	Apache 2.0	Powers ~30% of all websites globally
MySQL	Database	GPL v2 / Commercial	A license with a dual-model story
Firefox	Web Browser	MPL 2.0	A nonprofit fighting for an open web
VLC Media Player	Multimedia	LGPL / GPL	Plays anything — built by students in Paris
LibreOffice	Office Suite	MPL 2.0	Born from a community fork — a real lesson
Python	Programming Language	PSF License	A language shaped entirely by community
Git	Version Control	GPL v2	The tool Linus built when proprietary failed him

Your Choice	**Git — GNU General Public License v2 (GPLv2)**

3. Project Report

PART A — Origin and Philosophy (Units 1 & 2)

A1. The Problem Git Was Created to Solve

Git was created in April 2005 by Linus Torvalds — the creator of Linux — to manage the development of the Linux kernel. Its creation was triggered by a crisis in the Linux kernel community, driven by three key factors:

- Licensing Dispute: Since 2002, the Linux kernel project had used a proprietary version control system called BitKeeper. In 2005, the relationship with BitKeeper's parent company broke down and the free-of-charge status for open-source projects was revoked.
- Lack of Alternatives: No existing free VCS (CVS, SVN, etc.) met the Linux project's extreme requirements for speed, scale, and a truly distributed workflow.
- Clear Design Goals: Torvalds needed a system capable of handling thousands of parallel branches, protecting data integrity against corruption, and processing patches in seconds rather than minutes.

How Git Was Developed

- Torvalds began on 3 April 2005, building Git as a 'content-addressable filesystem' — fundamentally different from traditional source-control managers.
- By 7 April — just four days later — Git was self-hosting: the project was tracking its own source code using itself.
- Within one month, Git was benchmarked at 6.7 patches per second against the Linux kernel, meeting its ambitious performance goals.
- In July 2005, Torvalds handed maintainership to Junio Hamano, who led the project to its 1.0 release in December 2005 and remains the primary maintainer today.

A2. The License — What It Actually Says

Git is released under the GNU General Public License v2 (GPLv2) — a strong copyleft license that grants all four FSF freedoms:

Freedom	Guarantee	Git's Compliance
Freedom 0 — Run	Use the program for any purpose	Yes — including full commercial use
Freedom 1 — Study / Modify	Access and alter the source code	Yes — full source code is public
Freedom 2 — Share	Redistribute exact copies freely	Yes — freely redistributable
Freedom 3 — Improve / Distribute	Share modified versions with others	Yes — under GPLv2 (copyleft enforced)

Copyleft Protection: GPLv2 mandates that any derivative work distributed externally must also be released under GPLv2. A company may modify Git for internal use without disclosing changes, but any external distribution triggers the share-alike obligation. Selling a modified version without releasing the modified source is not legally permitted.

GPL vs MIT — A Practical Comparison

Aspect	GPL v2	MIT
Core philosophy	Copyleft — freedom must propagate	Permissive — maximum flexibility
Derivative works	Must remain open-source	Can be made proprietary
Commercial use	Allowed with conditions	Fully unrestricted
Internal modifications	Private is fine; distribution triggers GPL	Always private — no obligation to share
Best for	Community infrastructure & shared tools	Libraries seeking broad adoption

Personal recommendation: MIT for most personal projects on GitHub to encourage maximum adoption without legal constraints; GPL when the goal is to mandate that all future improvements remain open to the community.

'Free as in Beer' vs 'Free as in Freedom'

Feature	Free as in Beer (Gratis)	Free as in Freedom (Libre)
Focus	Zero monetary cost	Rights and liberty
Source code	Usually hidden	Always accessible
Modification	Restricted	Allowed
Redistribution	Restricted	Allowed
Examples	Adobe Reader, Skype	Linux, Firefox, Git

Git embodies both: it is free to download and use (gratis) and grants every user the right to study, modify, and redistribute it (libre). It is 'free and open source' in the fullest sense of both words.

A3. The Ethics of Open Source

Open source is not merely a licensing model — it is a statement about how knowledge and tools should be shared. The central debate balances transparency and collaborative innovation against intellectual-property protection and monetisation.

Arguments in Favour of Open Source:

- Transparency & Security: Open code allows anyone to audit for bugs and vulnerabilities — often yielding faster patches than closed-source alternatives.
- Innovation & Collaboration: Developers build on existing work, reducing redundant effort and accelerating progress globally.
- Freedom from Vendor Lock-in: No dependency on a single company for updates, pricing changes, or continued support.
- Customisation: Software can be adapted to specific needs without restriction or licensing fees.

Arguments Against Mandating Open Source for All Software:

- Funding & Sustainability: Developing high-quality software is expensive; proprietary models allow developers to recoup costs and invest in quality.
- Ease of Use: Commercial applications often provide more polished, user-friendly interfaces than community-driven OSS.
- Dedicated Support: OSS may lack the 24/7 technical support and professional documentation found in enterprise software.
- Intellectual Property: Companies may legitimately protect proprietary algorithms and competitive trade secrets.

Is it ethical for companies to profit from open-source community labour?

Perspective	Argument	Examples
Ethical — Legal	OSS licenses (MIT, Apache, BSD) explicitly permit commercial use	Red Hat, AWS, Google Cloud
Ethical — Value add	Companies provide support, security hardening, and infrastructure beyond base code	Red Hat RHEL model
Unethical — Exploitation	Massive extraction without support leads to burnout and security crises	Log4j, XZ Utils incidents
Unethical — Bait & Switch	Open source used for growth, then source is closed once successful	Various SaaS 'open core' pivots

Conclusion: Building a profitable product on open-source foundations is not inherently unethical, but it becomes exploitative when corporations treat community labour as a free resource rather than a collaborative partnership. Ethical behaviour means contributing code back, supporting maintainers financially, and strictly respecting license conditions.

PART B — Linux Footprint (Unit 2)

Installation Command

```
sudo apt install git
```

Key System Locations

Path	Purpose
/usr/bin/git	Main executable binary
/etc/gitconfig	System-wide Git configuration (all users)
~/.gitconfig	Per-user Git configuration
/usr/share/doc/git/	Documentation, changelog, examples

User & Permissions Model

Git runs as the currently logged-in user — it does not install a dedicated system service or daemon. This design ensures security boundaries are maintained: Git cannot escalate privileges beyond the invoking user, and no background process runs with elevated permissions.

Updating Git

```
sudo apt update && sudo apt upgrade
```

PART C — The FOSS Ecosystem (Units 3 & 4)

Core Dependencies

Dependency	Role
libc (GNU C Library)	Core system calls and C runtime — the foundation of all Linux programs
OpenSSL / libssl	Cryptographic operations; powers SSH and HTTPS transport
curl / libcurl	HTTP and HTTPS remote communication for git clone, fetch, push
zlib	Data compression for packfiles — critical for efficient storage and transfer
libpcre2	Regular expression support for git grep and pattern matching

Tools Inspired by Git

- GitHub — the world's largest code-hosting platform, built entirely around Git
- GitLab — a self-hostable alternative with integrated CI/CD pipelines
- Bitbucket — Atlassian's Git-hosting solution tailored for enterprise teams
- Gitea — a lightweight, open-source, self-hosted Git service
- Gerrit — a code-review tool used internally by Google and Android

Role in the Web / LAMP Stack

- Manages source code for web applications deployed on Apache or Nginx servers
- Enables automated deployment pipelines — push-to-deploy workflows via hooks
- Tracks database migration scripts alongside application code for consistency
- Integrates with CI/CD tools (Jenkins, GitHub Actions) for automated testing and release

Community & Governance

- Maintained under the umbrella of the Linux Foundation
- Primary maintainer: Junio Hamano (Gitster) — since July 2005
- Contributions accepted via the public git mailing list (git@vger.kernel.org)
- Thousands of contributors worldwide; no single corporate owner controls the project

PART D — Open Source vs Proprietary (Critical Analysis)

Side-by-Side Comparison: Git vs GitHub Enterprise

Dimension	Git (Open Source)	GitHub Enterprise (Proprietary)
Cost	Free	Paid subscription per user
Source Code Audit	Fully transparent	Limited / closed source
Security	Community-audited by thousands	Vendor-controlled audits
Support & Reliability	Community + third-party vendors	Dedicated SLA-backed support
Freedom to Modify	Unrestricted under GPLv2	Restricted by vendor EULA
Control Model	Decentralised — no single owner	Centralised — vendor-controlled
Customisation	Full — fork and modify freely	Limited to vendor-provided features
Vendor Lock-in	None	High — data migration is complex

Verdict & Recommendation

Open-source solutions like Git excel in transparency, flexibility, and cost-effectiveness. Because the code is fully public, security vulnerabilities can be discovered and patched by the global community — often faster than in closed-source products. Git's decentralised model ensures no single company can unilaterally alter its direction, revoke access, or impose pricing changes.

Proprietary platforms like GitHub Enterprise offer genuine value in dedicated support, reliability SLAs, and integrated enterprise tooling. For regulated industries or large organisations that require contractual guarantees, this trade-off can be justified.

Recommended approach: Use Git as the foundational version-control layer, and layer enterprise platforms (GitHub Enterprise, GitLab EE) selectively where workflow integration, compliance requirements, or support SLAs demand it. Contributing back to upstream open-source projects remains both a professional best practice and an ethical responsibility.

Personal commitment: Yes — actively contributing to open-source projects like Git is valuable not only for personal and professional growth but for strengthening the global developer community and making technology more accessible to everyone.

4. Shell Script Tasks

Five Bash scripts were developed to demonstrate practical command-line mastery and open-source skills. Each script is presented with its purpose, key techniques used, and the complete, exact source code as submitted.

Script 1 — System Identity Report (Units 1 & 2)

Purpose:

Collects and displays live system information alongside student and software metadata in a clean formatted report.

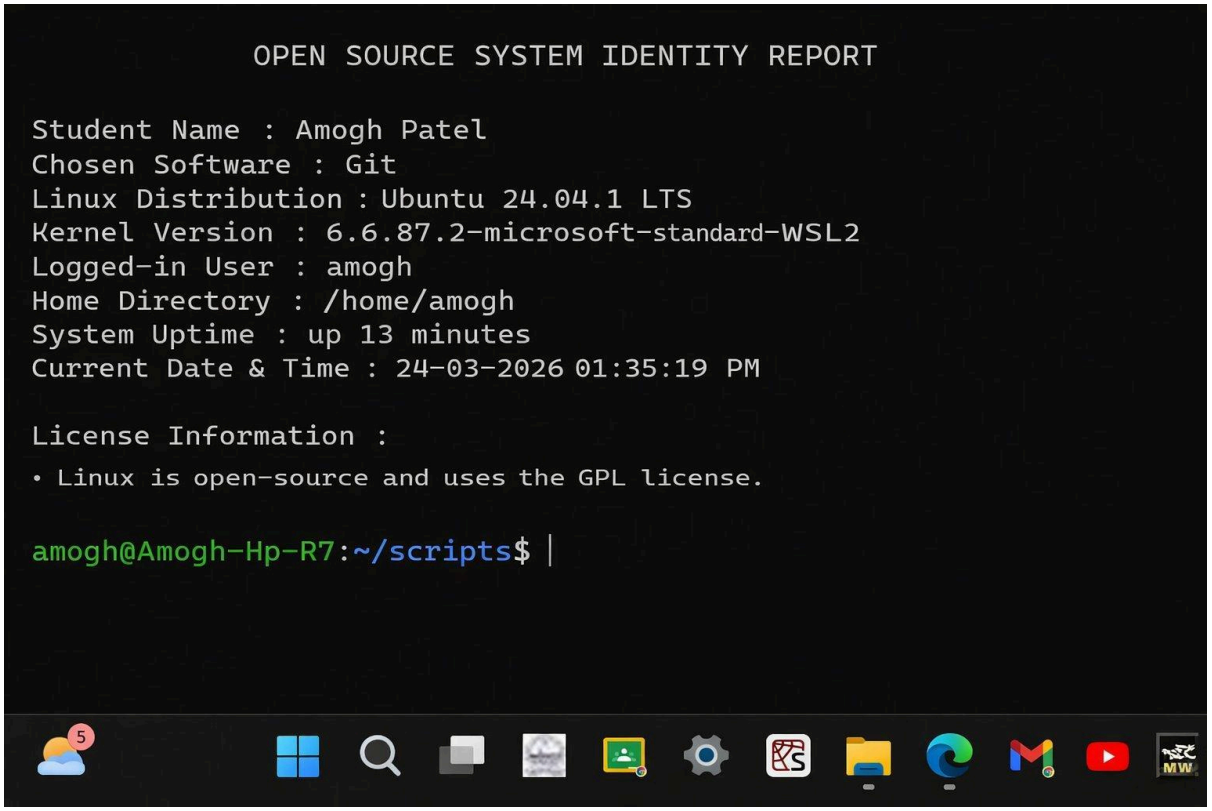
Key Techniques:

- Variable assignment for student metadata (STUDENT_NAME, SOFTWARE_CHOICE)
- Command substitution: uname -r, whoami, uptime -p, date
- Conditional file-read: checks /etc/os-release for distro detection
- String interpolation in echo statements for formatted output

```
#!/bin/bash

#Script-1:System Identity Report
#Author:Amogh Patel
#Course:Open Source Software

#variables(student/project information)
STUDENT_NAME="Amogh Patel"
SOFTWARE_CHOICE="Git"

#system information
KERNEL_VERSION=$(uname -r)    #kernel version
USER_NAME=$(whoami)   #current logged in user
UPTIME=$(uptime -p)   #uptime of system in readable format
HOME_DIR="$HOME"    #home directory of current user

# Current date and time
CURRENT_DATE_TIME=$(date "+%d-%m-%Y %I:%M:%S %p")
```

```
# Linux distribution name
if [ -f /etc/os-release ]; then  #we are identifying linux distribution details
    DISTRO_NAME=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d= -f2 | tr -d
'"')
else
    DISTRO_NAME="Unknown Linux Distribution"
fi

# License message
OS_LICENSE_MESSAGE="Linux is open-source and uses the GPL license."

#display
echo " "
echo "                OPEN SOURCE SYSTEM IDENTITY REPORT            "
echo " "
echo "Student Name  : $STUDENT_NAME"
echo "Chosen Software : $SOFTWARE_CHOICE"
echo "Linux Distribution : $DISTRO_NAME"
echo "Kernel Version : $KERNEL_VERSION"
echo "Logged-in User : $USER_NAME"
echo "Home Directory : $HOME_DIR"
echo "System Uptime : $UPTIME"
echo "Current Date & Time : $CURRENT_DATE_TIME"
echo " "
echo "License Information :"
echo "$OS_LICENSE_MESSAGE"
```

Output:



Script 1 Terminal Output

Script 2 — FOSS Package Inspector (Unit 2)

Purpose:

Checks whether the chosen FOSS package (Git) is installed, reports its installed version, and uses a case statement to display a philosophy note about the package.

Key Techniques:

- dpkg -l with grep -w to check installation status silently
- git --version for live version retrieval
- case statement for package-specific philosophy messages
- Output redirection to /dev/null to suppress noise

```
#!/bin/bash

#Script-2: FOSS Package Inspector
#Author:Amogh Patel
#Course:Open Source Software

#package name
PACKAGE="git"

#we will check if package is installed or not
if dpkg -l | grep -w "$PACKAGE" > /dev/null 2>&1; then
    echo "$PACKAGE is installed."
    dpkg -l | grep -w "$PACKAGE"
    echo "Version information:"
    git --version
else
    echo "$PACKAGE is NOT installed."
fi

echo ""

#using case statement for package description
case $PACKAGE in
    git)
        echo "Git: an open-source version control system created to track code
changes efficiently.";;
    apache2)
        echo "Apache: the web server that helped build the open internet.";;
    mysql-server)
        echo "MySQL: an open-source database used in many real-world
applications.";;
    firefox)
        echo "Firefox: an open-source browser that supports an open and
accessible web.";;
    vlc)
        echo "VLC: an open-source media player that can play almost any audio or
video format.";;
    *)
        echo "No philosophy note available for this package.";;
esac
```

Output:

```
amogh@Amogh-Hp-R7:~/scripts$ nano script2.sh
amogh@Amogh-Hp-R7:~/scripts$ chmod +x script2.sh
./script2.sh
git is installed.
ii git           1:2.43.0-1ubuntu7.1        amd64
ii git-man       1:2.43.0-1ubuntu7.1        all
ages)
ii lshw          02.19.git.2021.06.19.996aaad9c7-2build3 amd64

Version information: git version 2.43.0      amd64

Git: an open-source version control sysstem created to track code changes
efficiently.

amogh@Amogh-Hp-R7:~/scripts$
```

Script 2 Terminal Output

Script 3 — Disk and Permission Auditor (Unit 2)

Purpose:

Iterates over critical system directories, reporting permissions, ownership, and disk usage for each. Also checks for a Git configuration directory in the home folder.

Key Techniques:

- Bash array declaration for directory list
- for loop with ${DIRS[@]} to iterate all elements
- ls -ld piped to awk to extract permissions and ownership
- du -sh for human-readable directory size
- Conditional check for .git directory existence

```
#!/bin/bash

#Script-3: Disk and Permission Auditor
#Author:Amogh Patel
#Course:Open Source Software

#list of directories to check
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo " "
```

```
echo "        Directory Audit Report           "
echo " "

#doing loop through directories
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done

echo " "
echo "Checking Git configuration directory"
echo " "

#checking git config directory
if [ -d "$HOME/.git" ]; then
    ls -ld "$HOME/.git"
else
    echo "Git configuration directory not found in home"
fi
```

Output:

```
amogh@Amogh-Hp-R7:~/scripts$ nano script3.sh
amogh@Amogh-Hp-R7:~/scripts$ chmod +x script3.sh
./script3.sh
    Directory Audit Report

/etc => Permissions: drwxr-xr-x root root | Size: 4.2M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 73M
/home => Permissions: drwxr-xr-x root root | Size: 60K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 92M
/tmp => Permissions: drwxrwxrwt | Size: 20K

Checking Git configuration directory

Git configuration directory not found in home

amogh@Amogh-Hp-R7:~/scripts$ |
```

Script 3 Terminal Output

Script 4 — Log File Analyzer (Units 2 & 5)

Purpose:

Accepts a log file path and optional keyword as command-line arguments. Counts the total number of keyword occurrences line by line, then displays the last five matching lines.

Key Techniques:

- Positional parameters ($1, $2) with default value fallback via ${2:-"error"}
- File existence check with [! -f] and empty-file check with [! -s]
- while IFS= read -r loop for safe, line-by-line file processing
- grep -iq for case-insensitive keyword matching
- Arithmetic expansion $((COUNT + 1)) for incrementing the counter
- grep | tail -5 to display only the last five matching lines

```
#!/bin/bash

#Script-4: Log File Analyzer
#Author:Amogh Patel
#Course:Open Source Software

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

#checking if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

#checking if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "The file is empty."
    exit 1
fi

#reading file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
amogh@Amogh-Hp-R7:~/scripts$ nano script4.sh
amogh@Amogh-Hp-R7:~/scripts$ chmod +x script4.sh
./script4.sh
Error: File not found.
amogha@aAvinash-Hp-R7:.scipt4.sh script2.sh git
amogha@aAvinash-Hp-R2:cd scripts
./script4.sh script2.sh script2.sh git
Keyword 'git' found 4 times in script2.sh

Last 5 matching limes:
PACKAGE="git"
    git --version
    git)
        echo "Git: an open-source version control system created to track code
changes efficiently.";;

amogh@Amogh-Hp-R7:~/scripts$
```

Script 4 Terminal Output

Script 5 — Open Source Manifesto Generator (Unit 5)

Purpose:

Prompts the user with three reflective questions about their open-source values and writes a personalised manifesto to a text file named after the current user, then displays it on screen.

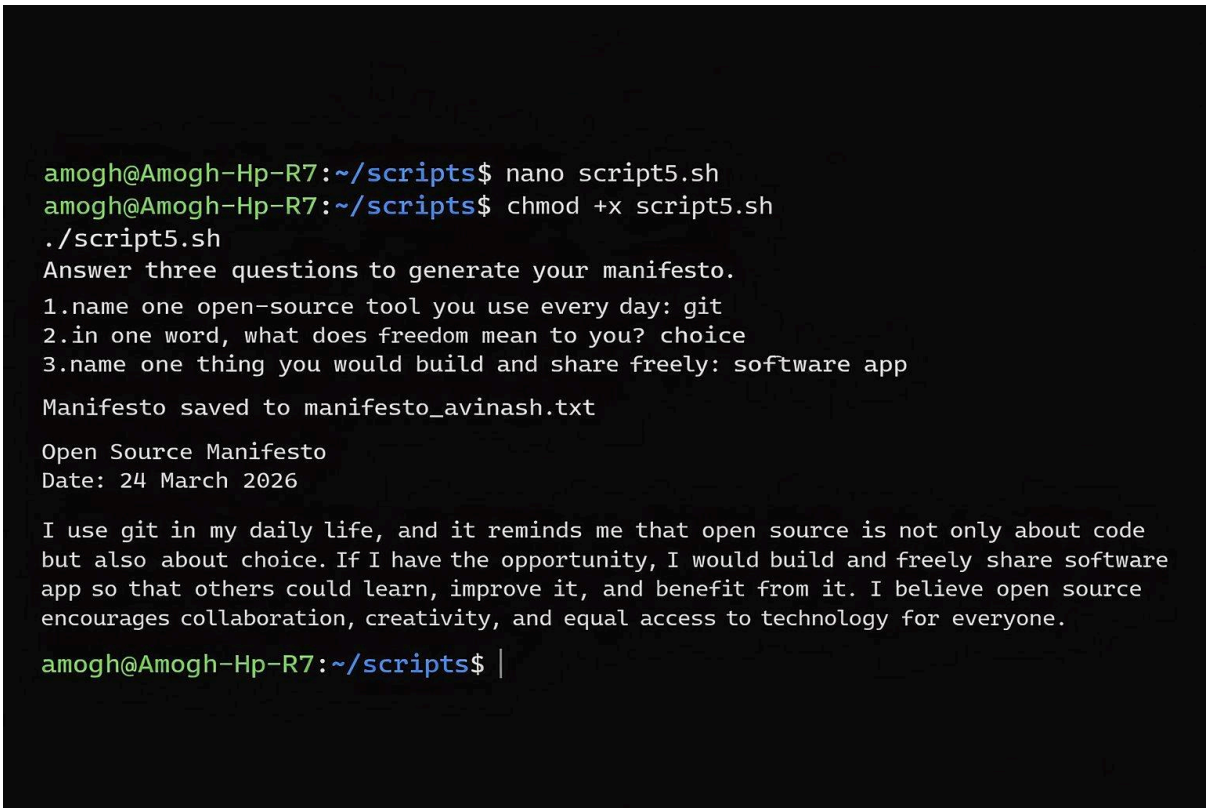
Key Techniques:

- read -p for interactive user input with inline prompts
- date '+%d %B %Y' for human-readable date formatting
- Command substitution $(whoami) to dynamically name the output file
- Output redirection: > to create the file, >> to append subsequent lines
- cat to display the completed manifesto to the terminal

```
#!/bin/bash

#Script 5: Open Source Manifesto Generator
#Author: Amogh Patel
#Course: Open Source Software

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1.name one open-source tool you use every day: " TOOL
```

```
read -p "2.in one word, what does freedom mean to you? " FREEDOM
read -p "3.name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

#example alias: alias ll='ls -l'

#writing manifesto to file
echo "Open Source Manifesto" > "$OUTPUT"
echo "Date: $DATE" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "I use $TOOL in my daily life, and it reminds me that open source is not
only about code but also about $FREEDOM.If i have the opportunity,i would build
and freely share $BUILD so that others could learn, improve it, and benefit from
it. I believe open source encourages collaboration, creativity,and equal access
to technology for everyone." >> "$OUTPUT"
echo " "
echo "Manifesto saved to $OUTPUT"
echo " "
cat "$OUTPUT"
```

Output:



Script 5 Terminal Output

5. Conclusion

The audit of Git as an open-source project illustrates how collaborative, community-driven development can produce tools that outperform proprietary alternatives in speed, reliability,

and longevity. Git emerged not from a corporate roadmap but from an immediate practical need — and that origin shapes its entire design philosophy.

The GPLv2 license ensures that Git's freedoms are self-perpetuating: every modification distributed to others must remain open, guaranteeing the global developer community can always inspect, improve, and build upon it. Understanding the distinction between 'free as in beer' and 'free as in freedom' is foundational to making informed choices as a professional developer.

Examining Git's position within the Linux ecosystem and the broader FOSS landscape highlights the interdependence of open-source projects. No tool exists in isolation; the libraries, standards, and communities surrounding Git are themselves open-source, forming a self-reinforcing ecosystem that powers the majority of the world's software infrastructure.

The ethical dimensions of open-source adoption remind us that using this ecosystem responsibly — contributing back, supporting maintainers, and respecting license conditions — is not merely good practice but an obligation to the communities whose labour underpins our work. Contributing to projects like Git is simultaneously an investment in one's own skills, one's career, and a stronger, more equitable technology landscape for everyone.

Amogh Patel | 24BAI10148
Open Source Software — A13 | 31 March 2026